

VDB_Checklist

What this is: every VectoraDB feature and its sub-features — built or not, and how it was tested — so it's clear where we are and what's next. How each feature works is in `VDB_Feature_Implemented.pdf` (same ids).

Last updated: 15 September 2026 · **Reflects:** `main @ 6a06175` + safety fixes batch 1 (`fix/safety-batch1`)

Source: `docs/VDB_Checklist.html` → `make feature-doc` → this PDF

✓

Done & tested — automated tests (unit, VM integration or CI)

🟡

Partial — gaps, or not verified end to end

✖

Known issue

□

Not started

Sub-features: ✓ built 🟡 partly □ not built yet ✖ broken · "VM \$n" = section of `make integration` ; "v2 \$n" =
`make integration-v2` ; "update" = `make integration-update`

Where we are

Area	Features	Done & tested	Done (manual)	Partial	Known issue	Not started	Progress
A · Install & setup	9	4	2	3	·	·	67%
B · CLI & services	2	1	·	1	·	·	50%
C · Branching & storage	8	6	1	1	·	·	88%
D · Durability	4	2	·	2	·	·	50%
E · Security foundations	3	2	1	·	·	·	100%
F · SQL gateway	7	6	1	·	·	·	100%
G · Accounts & auth	5	4	·	1	·	·	80%
H · Blackbox (record of schema changes)	18	14	·	4	·	·	78%
I · Import & migration	13	4	8	1	·	·	92%
J · ETL pipelines (experimental)	4	2	2	·	·	·	100%
K · APIs, SDKs & agents	6	1	2	3	·	·	50%
L · Web console	10	·	5	5	·	·	50%
M · Build, CI & releases	6	4	1	1	·	·	83%
Total (features built or started)	95	50	23	22	·	·	77%
N · Not started (roadmap)	11	·	·	·	·	11	0%

Storage engine — not implemented, by design

VectoraDB does **not** have its own storage engine. Data is stored by **PostgreSQL 16's own engine** (heap tables, B-tree indexes, write-ahead log) running on local disk inside the stock `postgres:16-bookworm` image plus `wal-g`. This was a locked architecture decision: a Neon-style engine that separates compute from storage over the network adds latency to every transaction, and transaction speed must never be compromised.

What **is** built is a storage *layer* around Postgres: copy-on-write branch drivers — ZFS on macOS/Linux, btrfs on Windows — behind one interface in `internal/branch/storage.go` (C1), quotas (C6), capacity reporting (C7), and durability to object storage with `wal-g` + MinIO: WAL archiving, base backups, point-in-time restore (D1–D3). Networked storage disaggregation is listed in the README roadmap but not started (see "Not started").

Suggested next steps

- Run `vdb update` against a real release on macOS and Windows (A9) — cut v0.9.0, install it, then update to v0.9.1.
- HA beyond the demo: archive WAL while main runs on the promoted standby, then automatic failover and fencing (D4). (*Safety fixes C3, D4 hazards, H7, I5, I6 done.*)
- Verify downloads everywhere: installers and `vdb setup` 's engine download still skip `SHA256SUMS` (A1, A2, A7).

4. Check the web pages that were built but never used signed in: Integrity, Policies, sidebar (L8–L10); add Blackbox filters (L5); refresh Guide/Docs (L7).
5. Add automated tests where there are none: imports (I1–I3, I5, I7, I8), SDKs (K3, K4), web UI (L*); bring the OpenAPI spec in line (K2).
6. Run the VM integration suites in CI (M2).

A Install & setup

6 of 9 done

ID	Feature · sub-features	Tested by
 A1	One-line installer (macOS / Linux) ✓ Detects OS / CPU, downloads the release binary ✓ Installs to the prefix (sudo only if needed) ✓ Stages the Linux engine for the VM on macOS	Manual
 A2	Windows installer (install.ps1) ✓ Installs WSL, elevates, resumes after reboot ✓ Downloads vdb.exe, engine, image build context ✓ Ubuntu roots verified against Ubuntu's checksums	Pester unit tests (Windows CI)
 A3	vdb setup on macOS (Lima) ✓ Creates / starts the Lima VM ✓ Installs Docker and ZFS ✓ Reinstalls the engine on every run ✓ Starts the stack	Manual (from-scratch VM run)
 A4	vdb setup on Windows (WSL2 + btrfs) ✓ Imports a dedicated distro with systemd ✓ Docker, btrfs and a storage mount unit ✓ Quiet steps, detail in install.log ✓ Waits for systemd on a cold distro	Unit (host_wsl_test.go); manual on Windows
 A5	Linux native ✓ Engine runs in-process, no VM	VM integration (all suites run on Linux)
 A6	Command forwarding & host file streaming ✓ macOS → Lima VM, starts the VM if stopped ✓ Windows → WSL distro	Unit (TestImportLocalFile , TestWSLArgs)
 A7	Always-latest engine binary ✓ Downloads the newest engine into the cache (ELF check)	Unit (host_test.go)
 A8	New-release notice on vdb start ✓ Background check, 1.5 s limit, printed last ✓ Only a promoted, newer, complete release	Unit; update \$1–\$3, \$8
 A9	vdb update ✓ Downloads and verifies everything before changing anything ✓ Stops servers, installs engine, restarts old one on failure ✓ up , Blackbox upgrade, start on the new engine	Unit; update \$4–\$7 (49 / 49)

B CLI & services

1 of 2 done

ID	Feature · sub-features	Tested by
 B1	The vdb command set ✓ Stack: setup, start, stop, up, down, status, logs, psql, update ✓ Branching, backup / restore, HA ✓ Blackbox, policy, impact	Unit (main_test.go , update_test.go); VM suites
 B2	Background services (daemon) ✓ Control plane :8080, gateway :6432, agent API :8088 ✓ Detached, pidfiles and logs in ~/.vectoradb	Unit (TestStopTerminatesProcess); VM \$1

C Branching & storage

7 of 8 done

ID	Feature · sub-features	Tested by
 C1	Storage drivers (ZFS / btrfs) ✓ ZFS snapshot + clone per branch (macOS VM, Linux) ✓ btrfs subvolume snapshots (Windows) ✓ Pool / filesystem created on a loopback file ✓ One storage interface for both 🟡 Repeated <code>reset --from</code> a non-main parent	VM §2 (ZFS); btrfs manual on Windows
 C2	Branch lifecycle ✓ Create, list, delete ✓ Reset to parent ✓ Suspend / resume ✓ Recreate a container from its surviving data ❑ Name validation in the CLI	Unit; VM §2, §4
 C3	Protection of main ✓ Delete and reset refused ✓ Suspend refused for main, the HA standby and whichever container serves main	Unit; VM §4, §6
 C4	Private networking (no host ports) ✓ Branch databases reachable only through the gateway 🟡 Exceptions: <code>restore :5433</code>, standby, MinIO	Unit (<code>TestParsePublishedPort</code> , <code>TestDSN</code>)
 C5	Scale-to-zero & wake on connect ✓ Idle branches suspended automatically ✓ Next connection wakes the branch (~0.3–0.7 s) ✓ HA standby never suspended ❑ Idle clocks survive a gateway restart	VM §7
 C6	Storage quotas ✓ Space reserved for main (ZFS) 🟡 Per-branch cap (opt-in) ❑ Quotas on btrfs	Manual
 C7	Capacity & status ✓ Services, main health, backups, branches ✓ Storage used / available 🟡 btrfs "used" is apparent size	VM §1
 C8	Provisioning & image pull ✓ Docker, storage pool and image prepared on <code>up</code> ✓ Pulls from GHCR, builds locally as fallback	Manual

D Durability

2 of 4 done

ID	Feature · sub-features	Tested by
 D1	WAL archiving to object storage ✓ wal-g ships main's WAL to MinIO, off the commit path ✓ MinIO images pinned on quay.io ❑ Branches archive their own WAL	VM §3
 D2	Base backups ✓ Create and list ❑ Schedule and retention ❑ Follow the primary after a failover	VM §3
 D3	Point-in-time restore ✓ Restore to a timestamp or latest on <code>:5433</code> ✓ Per-install credentials ❑ Use a base backup older than the latest	VM §3; v2 §1a
 D4	HA standby & failover ✓ Streaming hot standby ✓ Manual failover, <code>main</code> rerouted ✓ Nothing deletes or stops the promoted standby ✓ <code>vdb up</code> brings it back, not the old main ✓ <code>vdb ha fallback</code>, keeping every write 🟡 No WAL archiving while main runs on the standby ❑ Automatic failover, fencing, multi-host	Unit; VM §6, §7

E Security foundations

3 of 3 done

ID	Feature · sub-features	Tested by
 E1	Per-install generated secrets ✓ Generated Postgres and MinIO passwords ✓ Used by restore too	Unit (indirect); v2 §1a

✓	E2 TLS everywhere ✓ Self-signed TLS on all three services, zero setup	🟡 TLS optional at the gateway (<code>sslmode=disable</code> still connects)	Unit (<code>TestReadStartupUpgradesTLS</code>)
✓	E3 Zero-signup local API key ✓ First start creates a user, key and connection string	🟡 Windows setup summary doesn't show it	Manual

F SQL gateway 7 of 7 done

ID	Feature · sub-features	Tested by
✓ F1 Routing by database name ✓ One endpoint :6432, database name = branch	☐ Connection pooling, rate limiting	Unit; VM §1, §6
✓ F2 API-key authentication ✓ <code>vdb_</code> key as the Postgres password	✓ Bad key rejected with a Postgres error	VM §1
✓ F3 Backend login (SCRAM) ✓ Cleartext, md5, SCRAM-SHA-256 ✓ Clients never see the database password	☐ Unit test for SCRAM	VM §1, §6, §8 (exercised)
✓ F4 Per-user Postgres login roles ✓ Role per user, actor can't be forged	✓ Permissions of non-superuser <code>vdbclient</code>	Manual
✓ F5 Attribution session settings ✓ Actor kind, branch, session id per connection	🟡 Kind / branch / session client-settable	VM §8
✓ F6 Ctrl-C (CancelRequest) forwarding ✓ Cancel a running query through the gateway		Unit
✓ F7 Errors & insecure build tag ✓ Real Postgres error responses	✓ No-auth bypass impossible in release builds	Unit (<code>TestSendError</code>)

G Accounts & auth 4 of 5 done

ID	Feature · sub-features	Tested by
✓ G1 Auth store (SQLite) ✓ Users, keys, sessions, OAuth ids, pipelines ✓ Safe under concurrent use (WAL, busy timeout)	☐ Roles / permissions, multi-tenant projects	Unit (<code>TestVerifyKeyConcurrent</code>)
✓ G2 Email & password users ✓ Create account, log in ☐ Password change / reset	☐ List and delete users	Unit; VM §1
✓ G3 API keys ✓ Create (shown once), list, revoke ✓ Bearer / X-API-Key and gateway password	🟡 Revoking an unknown id reports success	Unit; VM §1
✓ G4 Web sessions & signup control ✓ <code>httpOnly</code> session cookie	✓ Signup can be closed	Unit (<code>TestSessions</code>)
🟡 G5 OAuth (GitHub / Google) ✓ Sign-in flows built ☐ Correct default URLs	☐ Tested with a real OAuth app	None

H Blackbox (record of schema changes) 14 of 18 done

ID	Feature · sub-features	Tested by
✓ H1 DDL recording ✓ Who, tool, branch, statement, status, risk ✓ Inherited by every branch	☐ Role / database changes, some dropped object types	VM §8

	H2 Hash chain, append-only & verify ✓ Rows can't be edited or deleted normally ✓ <code>vdb ledger verify</code>	🔗 Superuser rewrite caught only via anchors (H10–H12) ❑ Cover <code>vdb.policy</code>	VM §8; v2 §0
	H3 Attribution ✓ Humans via their login role ✓ Agents via their own <code>agent-<id></code> role	🔗 Tool, kind, session client-settable	VM §8; v2 §1c
	H4 Destructive-DDL guardrail ✓ DROP TABLE / DROP SCHEMA refused, attempt recorded	✓ Override only superusers / <code>vdb_admin</code>, use recorded ❑ "flag" and "allow" policy actions	VM §8; v2 §1b
	H5 Risk flags ✓ Type change, drop column flagged ❑ Per-command matching (not whole query text)	❑ Automated test	Manual
	H6 Blackbox access (CLI / REST / MCP / UI / SDK) ✓ CLI, REST with filters, MCP, web page, SDKs	❑ Per-user access control	v2 §0, §5
	H7 <code>vdb ledger revert --to</code> ✓ Help and output say it's a point-in-time restore of main	✓ Points to H13 for a branch from before a change	Unit (help text); VM §3 (the restore)
	H8 Override permission (<code>vdb_admin</code>) & upgrade ✓ <code>vdb admin grant revoke list</code> ✓ First user granted automatically	✓ <code>vdb blackbox upgrade [--all]</code>	v2 §1b
	H9 Blackbox 2.0 capture ✓ Transaction id and WAL position per entry ✓ Task, tool-call hash, override use	✓ Fail-safe, append-only ✓ Kill switch not settable by clients	Unit; v2 §2
	H10 Checkpoints & anchors ✓ Merkle root in the database and in read-only anchor files ✓ On a schedule and on demand	✓ Published anchor format ❑ Cloud write-once anchors (S3 Object Lock)	Unit; v2 §3, §3b
	H11 <code>vdb-verify</code> (independent verifier) ✓ Checks a live branch or an export	✓ Release binaries for every platform	Unit; v2 §3
	H12 Blackbox integrity & export ✓ INTACT / TAMPERED via CLI, REST, MCP ✓ JSON-lines export	❑ Streaming for very large ledgers	v2 §3
	H13 Branch from just before a Blackbox entry ✓ Restore + replay up to the change's transaction ✓ By transaction id or timestamp	✓ CLI, REST, MCP, web panel ✓ main untouched	Unit; v2 §4
	H14 Blackbox names & aliases ✓ <code>vdb blackbox</code>, <code>/blackbox</code> routes, MCP and SDK names	✓ Every original name still works	Unit; v2 §5
	H15 Policy gate (warn / block) ✓ VDB02 warn / VDB01 block with JSON detail ✓ Blocked attempts recorded ✓ Per-rule <code>vdb_admin</code> override	✓ CLI, REST, MCP, web page; custom rules 🔗 Patterns match the whole query text ❑ TRUNCATE	Unit; v2 §6
	H16 Agent provenance & <code>execute_change</code> ✓ Session, task, parent session on every agent change ✓ MCP <code>execute_change</code> with policy preview	✓ <code>vdb blackbox sessions</code>, REST 🔗 Provenance is what the agent says (not verified)	Unit; v2 §7

✓	H17 Impact analysis	✓ Dependents (views, foreign keys, indexes, depth) ✓ Other branches that have the object ✓ Policy verdict and risk score	✓ CLI, REST, MCP, inside <code>execute_change</code> 🟡 Functions, types, schemas not recognised from SQL	Unit; v2 §8
✓	H18 Blackbox diff between branches	✓ Fork point, changes only on each side ✓ Objects changed on both	✓ CLI, REST, MCP ☐ Merge (diff only today)	Unit; v2 §8

I Import & migration 12 of 13 done

ID	Feature · sub-features	Tested by
✓ I1	From Postgres ✓ Schema and data via <code>pg_dump</code> ✓ Zero-table result reported as an error	Manual
✓ I2	From MySQL ≤5.7 / MariaDB ✓ <code>pgloader</code> with type mapping (multi-arch image)	Manual
✓ I3	From MySQL 8.x ✓ Native <code>mysqldump</code> path (caching_sha2 auth) ✓ Types, NULLs, unicode, PK	Manual
✓ I4	From MongoDB ✓ Collection → typed table, exact name	VM §9, §10
✓ I5	From a .sql dump ✓ Runs the dump into a new branch ✓ Fails listing every failed statement (instance kept)	Unit; VM §11
✓ I6	From JSON / NDJSON ✓ Columns inferred from top-level keys	Unit; VM §10, §11
✓ I7	From CSV ✓ Table from the header, exact column names	Manual
🟡 I8	Continuous replication & cutover ✓ Initial copy + streaming (logical replication) ✓ Cutover keeps data (CLI)	Manual
✓ I9	Guard handling during import ✓ Guardrail paused, recording stays on	Manual
✓ I10	Live progress (CLI & SSE) ✓ Live log and progress bar	Manual
✓ I11	Import REST endpoints ✓ From a connection string or an uploaded file	Manual
✓ I12	Web Import page ✓ Database and file panels with live console	Manual
✓ I13	Exact identifier fidelity ✓ Case kept (<code>userId</code> , <code>leaseAiChats</code>)	Unit; VM §10

J ETL pipelines (experimental) 4 of 4 done

ID	Feature · sub-features	Tested by
✓ J1	Pipeline engine ✓ Extract → raw schema → SQL models → tests, in a fresh branch ✓ <code>ref()</code> / <code>source()</code> templating	Unit; VM §9

✓	J2 Data-quality tests ✓ not_null, unique, accepted_values, row_count_min, custom ✓ Failure → exit 1, data kept	Unit; VM §9
✓	J3 Stored pipelines & run history ✓ Saved per user, run over the API, history □ Read a run's log □ Mark runs whose process died	Manual
✓	J4 Pipeline editor (web) ✓ Create, edit, run, results, history □ Confirm before delete	Manual

K APIs, SDKs & agents 3 of 6 done

ID	Feature · sub-features	Tested by
✓	K1 Control-plane REST API ✓ Branches, query, Blackbox, policy, impact, import, pipelines, keys □ Versioned paths (/api/v1)	Unit; VM §1; v2 §0–§8
🕒	K2 OpenAPI spec ✓ Served, Blackbox 2.0 endpoints documented □ Match the code (import fields, status codes, missing routes)	Manual
✓	K3 Python SDK ✓ Branches, query, Blackbox □ Import, pipelines, keys, agents □ Tests □ Publish to PyPI	Manual
✓	K4 TypeScript SDK ✓ Same client for Node and browsers □ Tests □ Publish to npm	Manual
🕒	K5 Agent Branch API ✓ A branch per agent, own non-superuser role ✓ Provenance (task, parent, session) 🕒 DSN only reachable inside the VM 🕒 Cap and TTL count only running branches	VM §5; v2 §1c, §7
🕒	K6 MCP server ✓ Branches, run_sql, Blackbox tools ✓ policy_check, execute_change, impact, diff □ Authentication □ Client config snippet in docs	Unit; v2 §1c–§8

L Web console 5 of 10 done

ID	Feature · sub-features	Tested by
✓	L1 Embedded single-origin UI ✓ Served by the engine at :8080 □ Automated UI tests (whole area)	Manual
✓	L2 Login & signup ✓ Log in / sign up tabs, OAuth buttons	Manual
✓	L3 Dashboard ✓ Live tiles ✓ Branch table: create, suspend, resume, delete, copy DSN	Manual
✓	L4 SQL console & browser ✓ Schema sidebar, rows, structure, indexes ✓ SQL editor	Manual
🕒	L5 Blackbox page ✓ Entries per branch with badges □ Table / risk / date filters □ Verify button, statement and session	Manual
✓	L6 API keys page ✓ Create, copy, list, revoke	Manual
🕒	L7 Landing, Guide & Docs pages ✓ Landing, developer guide, reference □ Refresh stale content (https, sslmode, MCP, Blackbox, import routes)	Manual
🕒	L8 Blackbox integrity page ✓ Verify, checkpoint, export, branch-before □ Used in a browser against a running stack	Type-check + build

	L9 Policies page ✓ Rules, warn/block, custom rules, preview, evaluations □ Used in a browser against a running stack	Type-check + build; API in v2 §6
	L10 App sidebar layout ✓ Grouped sidebar, collapse, mobile drawer □ Reviewed in a browser while signed in	Type-check + build; served preview

M

Build, CI & releases

5 of 6 done

ID	Feature · sub-features	Tested by
 M1	Makefile targets ✓ build, vet, test, integration suites ✓ release, web, WSL builds, feature-doc (both PDFs)	CI
 M2	Linux CI ✓ Build, vet, test, gofmt ✓ Release script tests, checklist sync check □ VM integration suites in CI	CI
 M3	Windows CI ✓ Build, vet, test, Pester, installer encoding □ End-to-end WSL install test	CI
 M4	Releases & GHCR image ✓ Binaries for 5 platforms + vdb-verify ✓ SHA256SUMS merged safely between jobs ✓ Fails unless vdb update 's files are published ✓ Multi-arch Postgres image on GHCR	Script tests in CI
 M5	WSL distro & ZFS builds ✓ Prebuilt distro image □ Used by the installer □ Preloaded image tag fixed	Actions artifacts
 M6	Postgres image ✓ Postgres 16 + wal-g □ pgvector	Manual

Planned or discussed, nothing built yet. When work starts, move the row into its area with the next id there.

ID	Item · scope	Related
☐ -	Multi-tenant projects & roles ☐ Projects / organisations ☐ Per-branch permissions (RBAC)	G1, H6
☐ -	Branch merge ☐ Apply one branch's schema changes to another	H18
☐ -	Scheduled backups & retention	D2
☐ -	Automatic HA failover ☐ Health detection, fencing ☐ Multi-host	D4
☐ -	Pipelines: scheduling, incremental / CDC extract, DAG view	J1
☐ -	More import sources: Cassandra, DynamoDB	I1–I8
☐ -	pgvector in the Postgres image	M6
☐ -	Cloud write-once anchors (S3 Object Lock etc.)	H10
☐ -	Release signatures	A9, M4
☐ -	Hosted cloud tier	—
☐ -	Networked storage disaggregation (README roadmap; see the storage engine note on page 1)	C1, D1

Out of scope by decision

A custom storage engine (PostgreSQL's own engine is used, for native transaction speed), sharding, Raft / distributed transactions, multi-region, HTAP.